

杨会

求职意向：高级前端开发工程师

8年+前端开发经验

本科 · 1994.11

15910269957

yanghui2333@gmail.com

擅长前端基础设施与平台化建设，覆盖 SDK 体系、工程化构建、性能优化及跨端落地。

负责过多业务线公共能力沉淀与技术方案推进，能够在复杂业务场景中平衡复用性、稳定性与交付效率。

具备较强的跨领域协作与问题推进能力，可借助 AI 工具快速完成方案验证与原型落地。

专业技能

前端技术： React、Next.js、Redux、TypeScript、JavaScript

工程化构建： Webpack、Rollup、Vite、CI/CD (Jenkins)、多业务线构建体系

架构设计： SDK 体系设计、模块化架构、前端基础设施建设

性能优化： 首屏优化、懒加载与预加载、资源缓存、代码分割、虚拟列表、Web Worker

跨端与客户端： 微信小程序、Hybrid App、鸿蒙应用、JSBridge

数据监控与通信： 神策埋点、火山数据、性能监控、线上问题定位、WebSocket

AI 辅助开发： Cursor、Claude Code（用于方案设计、原型验证、代码生成、测试辅助）

工作经历

梦想科技 · 高级前端开发工程师

2019.05 — 至今

- 负责多个前端 SDK 的技术选型、架构设计与从 0 到 1 落地，稳定支撑多个线上项目
- 推进 React 项目的工程化架构设计与性能优化，搭建多业务线构建体系，支持多主题 / 多业务线动态构建
- 设计并落地配置驱动的多资方接入架构，以单一配置中心收敛三方机构差异化业务规则，核心链路以配置驱动为主，显著降低硬编码与分支散落
- 主导 AI 智能外呼系统方案设计与原型验证；落地公司首款鸿蒙混合 App 并完成上架
- 作为项目负责人牵头需求拆分、排期与跨端联调，推进核心模块落地；组织组内 Code Review 与技术分享，参与前端招聘面试与新人带教，推动规范沉淀

小猪先生科技有限公司 · 前端开发工程师

2018.07 — 2019.05

- 负责公司移动端项目开发，参与微信小程序开发与维护

宜聚（北京）网络科技有限公司 · 前端开发工程师

2017.05 — 2018.07

- 负责混合APP H5 侧业务开发，基于 WebView 容器实现核心业务页面，与原生端通过 JSBridge 完成交互对接
- 参与 PC 端管理后台开发及运营活动页面搭建，使用 React 完成功能模块的迭代与维护

项目经历

AI 智能外呼系统 — 前端跨栈主责 · AI 协同交付

2025 — 2026

面向电销/回访场景的自动化外呼需求，作为前端研发以 AI 协同方式完成"自动外呼 → 实时 ASR → 对话决策 → TTS"全链路系统的架构设计与核心实现，整体采用事件驱动 + 流式 Pipeline + 单通隔离架构。

核心工作

- **架构设计**：主导 SIP / ASR / LLM / TTS / 意图解析的模块拆分与职责边界；引入 **Generation ID 轮次标识** 统一处理打断（Barge-in）、过期音频淘汰与延迟动作；以事件 + 条件判断的隐式状态控制替代显式状态机，解决流式并发下的串话、错播与状态错乱
- **链路打通与流式优化**：打通 ASR 流式识别 → LLM 流式输出 → TTS 分段播放闭环，实现"边生成边播放"；ASR 终态断句 + 静音超时完成轮次判停；欢迎语预生成 + 流式切片降低首响应时延
- **工程化与稳定性**：FastAPI + RabbitMQ 支撑任务投递与并发消费；落地会话录音、日志与资源回收；补齐静音 / ASR 失败 / SIP 断连异常兜底；Prompt 远端缓存 + 本地降级提升弱网可用性
- **AI 协同**：主导需求拆解、模块划分、接口契约、关键时序与选型决策；以架构主导 + AI 协同编码的方式完成 Python 侧从 0 到 1 交付与内部试运行

SIP

WebSocket

ASR/TTS

FastAPI

RabbitMQ

LLM

pjsua2

Python

Cursor

前端 SDK 体系设计与落地

2020 — 2023

主导前端 SDK 体系规划与建设，以模块化、可插拔架构从 0 到 1 实现大数据采集、客户端交互、错误监控等核心 SDK，支撑多条业务线快速接入与长期演进。

核心工作

- **体系架构**：搭建模块化 SDK 体系，从 0 到 1 实现大数据采集（埋点上报）、客户端交互（JSBridge 通信）、错误监控等核心 SDK；各 SDK 职责内聚、独立发布、按需引入，隔离业务线交叉依赖，支撑多项目并行接入与独立演进
- **构建与发布**：基于 Rollup 搭建多规格构建流程，输出 UMD / ESM / CJS 三种产物并生成 TypeScript 类型声明；结合 Tree Shaking 与按需导出控制产物体积，保障轻量接入
- **工程化治理**：建立 SemVer 版本规范、Changelog 变更日志与规范化发布流程，保障版本可追踪、可回滚；结合线上监控与业务反馈持续打磨稳定性

TypeScript

Rollup

SDK 架构

模块化设计

项目工程化体系建设

2020 — 2025

为解决多业务线相似功能重复开发问题，基于 React 搭建平台化项目架构，支持多主题、多场景灵活配置，显著提升团队研发效率。

核心工作

- **多业务线构建体系**：基于 Webpack 深度定制构建配置，通过模块化设计支持多主题 / 多业务线动态构建，一套代码产出多条业务线产物，明显缩短新业务接入周期
- **弹窗调度体系**：底层封装命令式弹窗基础设施（\$.dialog 动态挂载 + React.lazy 按需加载 + 统一壳层与埋点切面），上层实现单例优先级队列调度器（支持优先级排序、霸屏抢占、批量注入与手动释放）；承载 60+ 弹窗组件、80+ 业务场景，首屏 bundle 零占用，彻底解决多业务弹窗并发下的抢占、遮挡与时序混乱。
- **配置化表单渲染引擎**：基于工厂模式设计声明式渲染器，以 JSON 结构描述驱动 10+ 种控件动态渲染，统一收敛显隐、校验、联动与多语言；新业务线接入仅改配置，页面零改动
- **多资方聚合配置中心**：采用策略模式 + 声明式配置，将 60+ 家三方机构在借还款、账单、费用、维护窗口等维度的差异抽象为 50+ 语义化字段；沉淀 20 种还款策略枚举，业务层彻底消除渠道级 if/else，新渠道接入工时由天级降至小时级
- **首屏性能优化**：通过资源整合、懒加载、预加载等策略，在内部测试中将首屏加载时间从 2.5s 优化至 **1.2s 以内**；自研 Webpack 插件，通过 html-webpack-plugin 的 afterTemplateExecution 钩子将 <link>/<script> 改为运行时注入，使内联 Loading 骨架先于主包渲染，白屏时间 ~800ms → ~200ms
- **CI/CD 与监控**：协助运维团队搭建基于 Jenkins 的前端自动化部署流水线，配置多环境构建参数及部署策略；落地 CDN 缓存策略，兼顾加载性能与上线可靠性；接入神策 / 火山数据平台，建立线上监控预警机制

React

Webpack

性能优化

Jenkins

CI/CD

神策数据

平台化架构

公司官网 Next.js 搭建与 SEO 基建

2023 — 2024

基于 Next.js App Router 搭建公司官网（品牌展示、博客资讯等模块），以 SSG / ISR 为主、SSR 为辅完成内容型页面渲染，并建立统一的 Metadata 与站点地图生成机制。

核心工作

- **渲染策略落地**：内容页采用 SSG + ISR（按内容更新节奏配置 revalidate），动态页走 SSR；合理划分 Server / Client Components 边界，将交互逻辑收敛到客户端组件，控制客户端 JS 下发体积
- **SEO 基建**：基于 generateMetadata 统一管理各页面 Meta 信息，集成结构化数据（JSON-LD）与 Sitemap / robots 自动生成，补齐官网 SEO 侧基础能力
- **资源与加载优化**：使用 Next.js Image 做图片懒加载与自适应尺寸；非首屏模块通过 dynamic import 按需加载，并结合 Suspense + loading.tsx 优化加载过程中的感知体验

Next.jsApp RouterSSGISRSEOReact

微信小程序开发与自动化部署

2020 — 2025

负责多个微信小程序的开发与维护，搭建自动化批量部署工具链，显著提升多端发版效率。

核心工作

- **自动化批量部署**：基于 miniprogram-ci 搭建自动化上传工具，支持多个小程序批量构建与上传；针对队列上传的状态污染问题，采用 child_process.fork 做进程隔离，显著提升多小程序发版效率

微信小程序miniprogram-cinode.js自动化部署多进程

鸿蒙混合 App

2024 — 2025

公司首款鸿蒙混合应用：H5 承载业务，原生侧以 ArkTS / ArkUI 集成 OCR、活体检测、登录等能力，封装 JSBridge 与网络层供 H5 调用，完成容器联调与上架。

ArkTSArkUIJSBridgeHybrid

教育背景

淮阴师范学院 · 地理信息技术 · 本科

2013.09 — 2017.06